Keelwater: product lifecycle diagnosis and 90-day plan

Andras Fugedi, 03/11/2026

Assumed (invented or from conversation): Gong, Fireflies, Intercom with Fin, FullStory, HubSpot, Notion, Shortcut, and Metabase on production Postgres; customer data stays in Keelwater's AWS VPC, audited; the platform's leading harness cannot run always-on agents; the factory launches next quarter; a head of data engineering is being hired; open-weight models are self-hosted; PMs can give a labelling half-day.

As build cost falls, the constraint moves left to deciding what to build and right to release and adoption. Coding and testing are about 25-35% of idea-to-launch time (Bain, FOUND[1]). Halving needs 50 points, so even zero build time leaves about 20 to find in Decide, Define and release (INFERRED), and section 6 shows my model does not find them within 18 months.

1. Where the lifecycle loses time and value

AI sped up the steps but not the joins, and the joins run through memory and meetings. The chain (INFERRED): feedback arrives by four routes nobody consolidates, so PMs hear the loudest account; decisions go unwritten, so settled arguments reopen and engineers and agents guess at intent; faster coding fills review queues; daily releases make customers retrain claims handlers.

- A third of PM time goes on status, data chasing and "what shipped?" (self-reported); each tool has its own metrics.
- Across 655 firms, coding agents raised PR volume about 36% and submission-to-approval time 56% (Harvard/Jellyfish working paper, FOUND).

Evidence missing: value-stream timing per squad, PR review wait, signal share per route, each tool's metrics side by side, retraining cost per release, a two-week PM time diary.

2. The lifecycle in an agentic world

The PDLC (product development lifecycle) runs from customer signal to measured outcome in seven stages. Agents read, draft and check; named people pass five gates.

Stage	Agents and pipelines	Gate (who)	Artefact
Sense	Nightly Signal Engine: triage, extraction, verified quotes	None; 40 drops audited weekly	Insights
Decide	Clustering, formula ranking, cited briefs, judges	Accept (PM)	Decision log entry
Define	Intent-doc agent; mockup generated from it on the real design-system components	Sign-off (Trio, designer included)	Intent doc, factory job
Build	Platform factory	Merge (engineers)	PR behind a flag
Release	Bundle assembler: notes, notices, customer impact	Bundle (PM, customer success)	Bundle manifest
Enable	Help articles, videos, post-call briefs	Comms (product marketing manager)	Articles, videos
Learn	Adoption against the intent doc's outcome metric	None	Outcome report

Handoffs. Each stage passes a typed artefact with a status; agents move items only into states their stage owns, only named people pass gates, and every move is logged. The intent doc and its factory job are the platform contract; the job cites evidence by reference, so quotes stay in the VPC.

Skills. Agent skills sit in one shared library: versioned, evaluated, owned by product ops, with squads adding their own. First five: intent-doc writer, release-notes drafter, SQL against the metric dictionary, call summary to insight, change-notice drafter. The human skill: using AI well.

Designers. Customer quotes and session-recording moments feed them directly; the designer signs off the generated mockup within the Trio gate.

The factory. A delegation meta-harness: signal in, it orchestrates people and sub-harnesses (the platform team's coding harness, Codex, a hosted or open-weight model) and passes the human gates. The delegation ruleset is its operating rules; owning the coding harness is not required.

Noise and accuracy. A sub-cent decision model drops chatter; 40 sampled drops a week measure what it misses. Code checks every quote, citation and number. Three judges from another model family (faithfulness, signal, duplication) stay "calibrating" until 50 blind PM labels per class put each one's true-positive and true-negative rates' 95% lower bound at 0.75.

Deploy daily, release in bundles. Code ships daily behind flags; customers get an announced monthly bundle with a preview, the controlled change the EU's Digital Operational Resilience Act, Art. 9 expects (FOUND). Customers wait up to four weeks, so I track idea-to-production-behind-flag and idea-to-customer separately and offer opt-in early access.

3. Shared infrastructure

Standard is whatever crosses squads, feeds a comparison or must survive an audit.

Standard (product ops owns)	Set by each squad
Signal schema and core taxonomy	Taxonomy extensions, ranking weights
Intent-doc template, factory job schema, decision log, skill library	Own skills added to the library, extra template sections
Model gateway and routing rules	Prompts within guardrails
Eval harness and judge registry	Judge thresholds above the shared floor
Telemetry, audit, access control	Digest cadence, in-squad visibility
One metric dictionary: v0 (about 10 lifecycle metrics) in 60 days, the rest later with data engineering	Squad metrics, defined in it

4. The first 90 days

Everything runs in the VPC on a product-ops job runner, moving to the platform harness if it schedules. My ask: one engineer for 90 days and platform support for the intake API; without the engineer, day 90 covers three squads, not five.

Day 30. Running: the Signal Engine nightly for a volunteer pilot squad on tickets and sales calls; a Shortcut "what shipped" digest for every squad. Users: the pilot Trio; all PMs for a labelling half-day. Learned: signal share per route, value-stream baselines. Go: the pilot PM accepts three of the top 10 in 30 minutes.

Day 60. Running: all routes plus sessions; intent-doc handoff for two squads; decision log; metric dictionary v0; flags and bundle manifest in the digest; a bundle pilot with three large customers. Users: two Trios, customer success. Learned: acceptance, judge agreement, review churn. Go: acceptance above 30%, no pilot customer leaves bundles.

Day 90. Running: intake for three to five squads, by staffing; outtake drafts (release notes, help articles, change notices, follow-up briefs). Users: PMs, product marketing, sales. Learned: pilot against control; where the bottleneck moved.

Adoption is the harder half:

- The VP Product sponsors it; the pilot squad volunteers.
- I give first: the digest removes status reporting before PMs label.
- Labelling happens in the Monday review, where the board replaces the weekly feedback meeting.
- Metrics: weekly active PMs, % of decisions with written rationale, % of customer-visible changes flagged.

5. What I will not touch, and the platform contract

I leave alone what another team owns and anything that would blur the pilot.

- The coding-agent harness is the platform team's; a second would split two engineers.
- Engineering owns code-review policy; I supply review-wait data.
- Squad structure: changing it with tooling would hide which worked.
- The design system itself; I only generate mockups on its components.
- The warehouse and metrics beyond v0, which come later with data engineering.

I ask the platform team for an intake API, a co-owned versioned job schema, gateway access with per-team cost tags, OpenTelemetry into self-hosted Phoenix, previews and demo videos, and per-customer flag cohorts. I own the lifecycle agents, job runner, schemas, evals, judges and adoption.

6. What to tell leadership and what to measure

Tell the board that halving idea-to-customer is not plausible within 18 months. In my halving model the mid case reaches 79% of today's lead time at month 18 (high 51%, AI alone 95%), decided mainly by the discovery-and-decision and release process effects, which are INFERRED, not measured. Coding speed alone cannot: in a McKinsey study of 40 PMs, a 40% productivity gain moved time to market about 5% (FOUND). Achievable: in 90 days, status work down from a third of PM time to under a tenth, feedback-to-decision in days, every build decision written; by month 18, about a fifth off lead time (INFERRED). A monthly board tracker reports lead time toward 50% and throughput toward 2x.

Before adding squads, compare pilot and control on both lead times, throughput, decision latency, release time, status hours, acceptance, judge agreement and review wait. My recommendation changes if:

- pilot decision time falls by half and release time by 70%: halving is in reach, so I add squads sooner;
- acceptance stays below 30% after calibration: rework taxonomy and ranking with PMs first;
- review dominates lead time: fund review capacity (smaller PRs, agent pre-review) first;
- customers reject bundles: give each customer flag-level control instead.

Part 2, the working Signal Engine, is in this repo; start at the README.

1. FOUND: in the cited source. INFERRED: my reasoning from sources.